Name:	**Muhammad Ammar Janjuha**
	Huzam Ayaz Khan
Registration No:	**BCS213104**
	BCS211087
Section:	**01**
Subject:	**Parallel & Distributed Computing**
Project:	**Map Reduce-Word Count**
Submitted to:	**Dr. Muhammad Siraj Rathore**
Date:	**04-January-2025**

Department of Computer Science

Contents

MapReduce Word Frequency Counter Using MPI 3
1. Main components and functionality 3
2. Counting words in a text 3
3. Master & Worker Nodes 4
4. Outputs: 5
Serial Version: 6
Parallel Version: 6
5. Performance and Speedup: 7
6. Conclusions 7
7. References 8

MapReduce Word Frequency Counter Using MPI

1. Main components and functionality

MapReduce is a programming model and data processing paradigm tailored for large-scale computations in distributed computing environments. It divides complex tasks into three main phases.

Map Phase

In this initial step, data is transformed into intermediate key-value pairs by a set of mappers. Each mapper processes a portion of the input data independently, emitting intermediate results.

Shuffle Phase

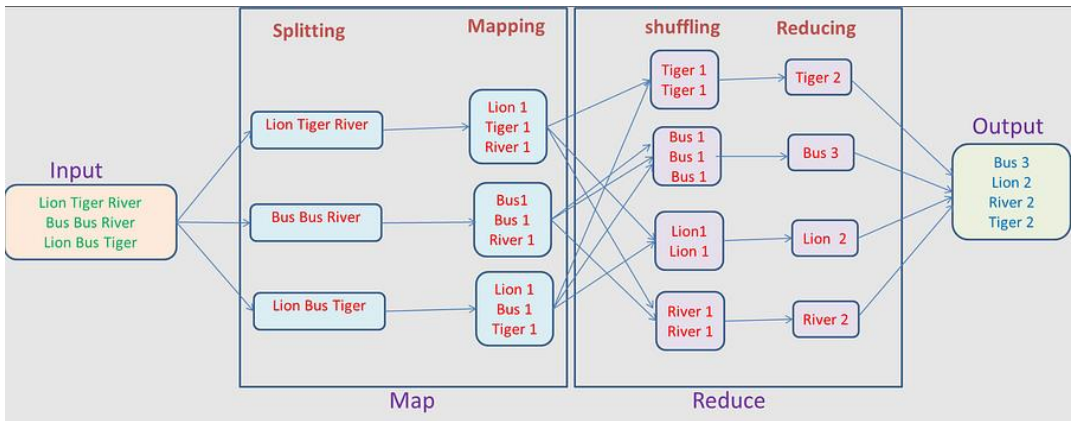
Following the Map phase, the intermediate key-value pairs are shuffled and distributed across the nodes in the computing cluster. This phase ensures that all values associated with a particular key are brought together, optimizing data distribution for the subsequent reduction.

Reduce Phase

The final step involves aggregating and processing the shuffled key-value pairs using a set of reducers. Reducers operate on grouped data, producing the ultimate output of the MapReduce job.

MapReduce excels in scalability, fault tolerance, and efficiency, making it a preferred choice for handling extensive datasets and parallel processing tasks across distributed computing nodes. It is widely applied in scenarios such as batch processing, log analysis, and large-scale data transformations within the realm of big data applications.

2. Counting words in a text



Let's examine the illustration depicting the process of tallying word occurrences in a text. On the left side, we have the input text: "Lion Tiger River Bus Bus River Lion Bus Tiger." This text is then divided into three parts for processing by separate workers.

The text is divided into three parts: “Lion Tiger River,” “Bus Bus River,” and “Lion Bus Tiger.” Each part is then passed to a Map function, which breaks down the text into intermediate key-value pairs.

For example, for “Bus Bus River,” we get the key-value pair “Bus — 1, Bus — 1, River — 1.” Essentially, we create key-value pairs with the word and assign that it occurred once.

Next is the Shuffle phase, during which all values associated with a specific key are consolidated. For instance, the occurrences of the word “Tiger” are consolidated together, and the occurrences of “Bus” are consolidated together.

Following this is the Reduce phase, where the grouped key-value pairs are combined and analyzed to determine the occurrences. In this example, there are three occurrences of “Bus.” When we output from the Reducer that “Bus” occurred three times, and “River” occurred twice, and so on.

Finally, data is gathered from the Reducers, and we produce the counts of each word, which are: “Bus — 3,” “Lion — 2,” “River — 2,” “Tiger — 2.”

3. Master & Worker Nodes

The MapReduce architecture is comprised of a Master machine and Worker machines. The Master and Workers are essential components of the distributed computing framework.

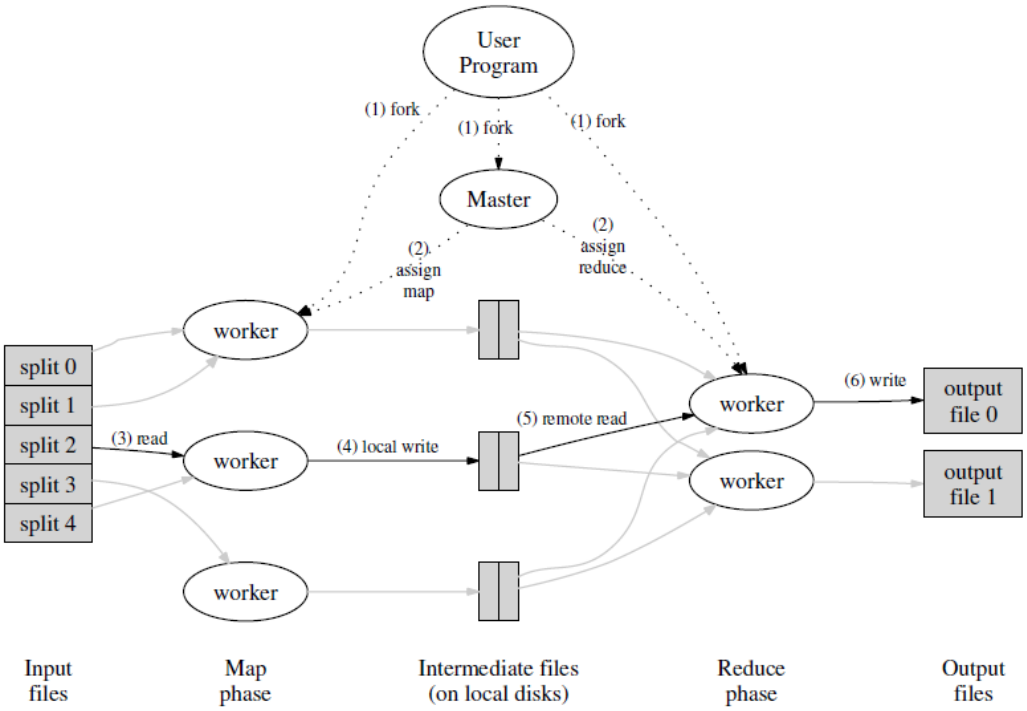
Master Node

- The Master Node is responsible for coordinating the entire MapReduce job.
- It manages the assignment of tasks to worker nodes, tracks their progress, and ensures the overall workflow is executed efficiently.
- The Master Node holds metadata about the input data, the intermediate data, and the final output.

Worker Nodes

- Worker Nodes are responsible for executing the actual computation tasks assigned to them by the Master Node.
- They process portions of the input data using the Map and Reduce functions.
- Worker Nodes communicate with the Master Node to report progress and request additional tasks.

In a nutshell, the Master Node acts as the central controller, orchestrating the flow of the MapReduce job, while the Worker Nodes perform the computation tasks in parallel on different portions of the data. This distributed approach enables the efficient processing of large datasets by dividing the workload across multiple nodes.



4. Outputs:

```
ammar-janjuha@ammar-janjuha-VMware-Virtual-Platform:~/PDC/Project MapReduce$ chmod +x setup.sh run_serial.sh run_parallel.
sh analyze_performance.sh run_all.sh
ammar-janjuha@ammar-janjuha-VMware-Virtual-Platform:~/PDC/Project MapReduce$ ./run_all.sh
================================================
  COMPLETE MAPREDUCE BENCHMARK
================================================

Setting up environment...
================================================
  MapReduce Word Count - Complete Setup
================================================

[INFO] Starting setup process...
[SUCCESS] MPI found: gcc (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0
[INFO] Compiling serial version...
[SUCCESS] Serial version compiled successfully!
[INFO] Compiling parallel version...
[SUCCESS] Parallel version compiled successfully!
[INFO] Sample files already exist.

================================================
[SUCCESS] Setup completed successfully!
================================================

Available scripts:
  1. ./run_serial.sh          - Run serial version only
  2. ./run_parallel.sh        - Run parallel version only
  3. ./analyze_performance.sh - Analyze performance

To run everything sequentially:
  ./run_serial.sh && ./run_parallel.sh && ./analyze_performance.sh
```

Serial Version:

```
Step 1: Running serial version...
==================================================
  SERIAL WORD COUNT EXECUTION
==================================================

Available input files:
-rw-rw-r-- 1 ammar-janjuha ammar-janjuha 362 Jan 16 01:01 input1.txt
-rw-rw-r-- 1 ammar-janjuha ammar-janjuha 790 Jan 16 01:00 input2.txt
-rw-rw-r-- 1 ammar-janjuha ammar-janjuha 206 Jan 15 22:58 input3.txt
-rw-rw-r-- 1 ammar-janjuha ammar-janjuha 587 Jan 16 00:54 input4.txt
-rw-rw-r-- 1 ammar-janjuha ammar-janjuha 621 Jan 16 01:05 large_input.txt

Select files to process:
1. Small files only (input1.txt input2.txt input3.txt)
2. Large file only (large_input.txt)
3. All files
4. Custom selection
Enter choice (1-4): 3

Processing files: input1.txt input2.txt input3.txt large_input.txt

Running serial word count...
----------------------------------------

=== SERIAL EXECUTION COMPLETE ===
Execution Time: .012983466 seconds
Total Words Processed: 282
Unique Words Found: 152
Results saved to: results/serial_results.txt
Performance data: results/serial_performance.txt
Full output: logs/serial_output.log
```

Parallel Version:

```
Step 2: Running parallel version...
==================================================
  PARALLEL WORD COUNT EXECUTION
==================================================

Available input files:
-rw-rw-r-- 1 ammar-janjuha ammar-janjuha 362 Jan 16 01:01 input1.txt
-rw-rw-r-- 1 ammar-janjuha ammar-janjuha 790 Jan 16 01:00 input2.txt
-rw-rw-r-- 1 ammar-janjuha ammar-janjuha 206 Jan 15 22:58 input3.txt
-rw-rw-r-- 1 ammar-janjuha ammar-janjuha 587 Jan 16 00:54 input4.txt
-rw-rw-r-- 1 ammar-janjuha ammar-janjuha 621 Jan 16 01:05 large_input.txt

Select files to process:
1. Small files only (input1.txt input2.txt)
2. Large file only (large_input.txt)
3. All files
4. Custom selection
Enter choice (1-4): 3

Select number of processes to test:
1. 2 processes only
2. 4 processes only
3. 2, 4, and 8 processes
4. Custom process counts
Enter choice (1-4): 2

Configuration:
  Files: input1.txt input2.txt large_input.txt
  Process counts to test: 4

----------------------------------------
```

5. Performance and Speedup:

```
Step 3: Analyzing performance...
==================================================
  PERFORMANCE ANALYSIS
==================================================

=== PERFORMANCE SUMMARY ===

SERIAL EXECUTION:
  Time: .012983466 seconds
  Total Words: 282

PARALLEL EXECUTION:
Processes | Time (s) | Speedup | Efficiency
----------|----------|---------|-----------
4         | .907022419| .01     | 0%


=== BEST CONFIGURATION ===
  Processes: 4
  Speedup: .01x (vs serial)
  Efficiency: 0%


=== SPEEDUP CHART ===
4   procs:  0.01x |
```

6. Conclusions

In conclusion, MapReduce is a crucial framework in distributed computing that has revolutionized how we handle large datasets. Its core components — the Map, Shuffle, and Reduce phases.

The architecture, which includes Master and Worker Nodes, effectively manages MapReduce jobs. Parallel processing, a key strength, ensures improved performance, scalability, and more.

In summary, the combined impact of MapReduce spans diverse domains, from batch processing to log analysis, ushering in a new era of computational efficiency and data processing excellence.

7. References

1. Map Reduce explained with example | System Design
2. https://youtu.be/MAJ0aW5g17c?si=Cm4xYDs_mmkZgfsi
3. mapreduce-osdi04.pdf (googleusercontent.com)
4. map-reduce-word-count-openmp-mpi/openmp.c at main · adhithadias/map-reduce-word-count-openmp-mpi · GitHub
5. (PDF) MapReduce for Counting Word Frequencies with MPI and GPUs
6. GitHub - adhithadias/map-reduce-word-count-openmp-mpi: This repository contains an implementation of counting words in many files using the map-reduce algorithm. The algorithm is implemented in both OpenMP and MPI. A serial implementation is also available for perf evaluation.